# Interpretable Experiential Learning Based on State History and Global Feedback

Anonymized Name Surname
*Anonymized dept*
*Anonymized organization*
Anonymized City, Anonymized Country
Anonymized ORCID

***Abstract*—A new interpretable experiential learning model based on state history and global feedback is presented. This model extends the concept of reinforcement learning by applying it to tasks in which reinforcement may not be provided explicitly or may be significantly delayed. The model is represented as a transition graph between sequences of states and individual states, where state variables are described by measurable properties of observed objects. The proposed solution is capable of learning a behavioral model represented as a transition graph between sets of states, wherein transitions are assigned utility values and evidence counters; this enables the use of joint utility and probability distributions for decision-making purposes. The principle of "global feedback" ensures efficient learning under conditions of delayed and sparse feedback, even when utilizing low-end hardware. Consequently, this solution becomes practically applicable to interpretable process mining tasks across various domains—including environments with limited computational resources. This model is expected to be suitable for addressing reinforcement learning tasks in resource-constrained environments; it has undergone comprehensive evaluation on the OpenAI Gym Atari Breakout benchmark, demonstrating results that surpass human performance and are comparable to those of several well-known neural network-based solutions.**

***Index Terms*—edge computing, experiential learning, explainable AI, global feedback, interpretable AI, low-end computing, process mining, reinforcement learning, psyche model, resource-constrained computing, state history**

## I. Introduction

Reinforcement learning (RL) technology has undergone rapid development over the past 10 years [1]–[3]. Significant advances have been made in virtual gaming environments, where deep reinforcement learning algorithms have demonstrated performance significantly superior to human capabilities [4]–[7].

However, the deep reinforcement learning-based methods used in the above-mentioned works are limited in terms of providing interpretability and trustworthiness with respect to the learned models, which remains problematic for the application of such methods in mission-critical industrial automation tasks [1], [8], [9] or even less critical home automation tasks. In [8] we find "Interpretability, explainability, and transparency are key issues to introducing artificial intelligence methods in many critical domains ... Reinforcement learning methods, and especially their deep versions, are such closed-box methods". In particular, in respect to industrial automation, [1] calls for "practical industrial applications ... emphasizing transparency, sustainability, and operational resilience".

At the same time, the "process mining" approach [10]–[12] can be implemented based on state transition graphs or—as we propose in this work—based on graphs of transitions from preceding sequences of states to subsequent states, featuring "situational forks" along the branches of scenario development, as described in [13]. Moreover, the accumulation of statistical data such as evidence counts regarding transition frequencies enables the conduct of Granger causality analysis [14] based on the extracted process trees.

Moreover, "smart home" automation applications [15]–[17], as well as other potential consumer electronics applications, can be considered examples of "edge computing," "resource-constrained computing" or "low-end computing," where computing power is limited and the use of GPU cards or even clusters of such cards for deep reinforcement learning is not feasible. [18] states the "prevalent challenges encountered in RL optimization, including issues related to sample efficiency and scalability; safety and robustness; interpretability and trustworthiness".

An additional problem, typical for real automation scenarios, is the sparse, delayed or implicit reward, which forces us to rethink the concept of "reinforcement learning" as a marginal case of "experiential learning" [3], [19], [20] where feedback from the environment may not necessarily be rewarding, and environmental observations can be used to infer implicit feedback, while both explicit and implicit feedback may be applicable over long periods of observation without temporal decay, according to "global feedback" principle in [19].

Another key challenge in this area is the need for dimensionality reduction in the form of representation learning [21], which reduces the high-dimensional space of the field's input data to a low-dimensional space of hidden or latent variables. In terms of the required interpretability [1], [8], [18], we can consider this low-dimensional space to be interpretable, represented by reasonable features, objects, and events, as shown in [19].

In this study, we focus on solving the problem of sparse or delayed reward mentioned above under interpretability constraints. In this context, the proposed solution creates interpretable representations of domain-specific problems through pre-processing that transforms time series of states of the

original field data into time series of interpretable objects, events, and properties associated with these objects. These interpretable states and their historical sequences then themselves become part of the inferred model without introducing additional hidden or latent states.

Since notable progress in reinforcement learning is being made using the OpenAI Gym Atari benchmarks [29], we evaluate our solution in the context of the Atari game "Breakout", referring to earlier experiments described in [4]–[7], [25]–[28]. The game "Breakout" was chosen as one of the most popular OpenAI Gym environments, offering an experience similar to a single-player version of a real "ping-pong" game.

## II. Related Work

### A. OpenAI Gym Atari Breakout Baselines

The use of OpenAI Gym Atari environments as a platform for developing reinforcement learning algorithms has become popular since [29]. In these environments, each game is modeled by a *step* function, which takes an *action* argument and returns a *observation* structure representing the changed state of the environment corresponding to the next visual frame, a *reward* value containing explicit feedback, *terminated* and *truncated* variables signaling the end of the game, and an *info* structure providing some additional information that can be used to obtain implicit feedback, such as game termination in case of the "Breakout". Each interaction with the *step* function corresponds to one frame of the actual video game if *frameskip=1* is set. During a game, the player either earns points by hitting bricks on the opposite side of the game field, moving the racket (paddle) with appropriate actions, or loses a life. If all bricks are destroyed, or 864 points are scored, or 5 lives are lost, the game ends. The number of points achieved at the end of the game is considered the game score. In the case of the "Breakout," the maximum number of steps or frames is 108,000. However, the scoring methodology used in [29] calls for the game to end after 18,000 frames (corresponding to 5 minutes of play) if it has not already been completed.

Quick progress in this area began with the work of [5], which applied the deep Q-learning (DQN) method. For the game "Breakout," an average score of 168 with a maximum possible score of 225 was achieved after 50 training epochs (corresponding to 5,400,000 training frames), with each epoch consisting of 30 minutes of play (108,000 frames). This study also evaluated the average score of 31 achieved by an expert human game player. Another baseline was achieved by implementing an extended version of the DQN algorithm called Rainbow in combination with the Implicit Quantile Networks (IQN) method [7], which takes into account probability distributions on state transition graphs during training. The new algorithm achieved average scores of 53.77, 121.83, 132.56, and 175.47 for training frames in the millions of 10, 50, 100, and 200, respectively.

Progress accelerated with the implementation of distributed multitasking capacity and history-based experience replay in the Recurrent Replay Distributed DQN (R2D2) architecture, achieving 837-864 scores under different experimental conditions [26]. The inclusion of implicit feedback (called intrinsic reward) was introduced in Never-Give-Up (NGU) architecture, based on extrinsic (explicitly provided by the environment) and intrinsic (implicitly derived) rewards (feedback) based on exploratory activity [28], achieving 576-864 scores across different evaluation conditions. Another multi-agent distributed RL architecture, Agent57, combined history-aware DQN, IQN, and NGU with exploratory capabilities and achieved a score of 790 [4]. A record score of 864 was achieved by the latest MuZero architecture, which added planning capabilities to other features [6], where planning was based on a hidden model of the environment learned through interaction with it.

### B. Interpretable Approaches

A review of earlier work on interpretable reinforcement learning approaches is presented in [30]. The most recent paper [31] presented the possibility of learning so-called "Behavior Trees" and evaluated the results on several classic OpenAI Gym environments, confirming the effectiveness of the method. Furthermore, the paper [32] presented the possibility of learning "Interpretable Continuous Control Trees" (ICCTs) and confirmed the practical application of the method in real field conditions.

In the earlier works on so-called "experiential learning" [19], [20], the feasibility of constructing models based on non-latent explicit representations of environmental states, either on high-dimensional raw field data (termed "discrete") or on interpretable low-dimensional representations of the environment in terms of real-world objects and events (termed "functional"), was investigated. In these studies, experiences—regardless of whether or not they were reinforced—are encoded as sequential episodic memories for subsequent retrieval and replay; this depends on the extent to which these sequential episodes receive reinforcement at the conclusion of the entire episode as a whole. A mathematical model of a single-player ping-pong game with rules similar to those of the OpenAI Gym Atari "Breakout" was used, and it was confirmed that the same learning model, based on the history of states with so-called "global feedback", can be used to achieve the same level of performance in the same environment using both of two representations under the same evaluation conditions. The only difference was that learning and execution based on high-dimensional "discrete" (raw field) data turned out to be much slower and computationally expensive than learning based on low-dimensional "functional" representation. "Global feedback" implied that the utility values of state transitions, corresponding to actions specific to each state, were updated not gradually, as in DQN, but rather over entire sequences of states in the agent's episodic memory, so that each state and action in the corresponding sequence was updated equally. The boundaries of the state sequences were determined by the moments of receiving either positive feedback (the ball hitting the paddle or the opposite wall) or negative feedback (the ball missing the paddle on the player's side).

The approach proposed above appears plausible from a neuro-physiological perspective regarding the process of episodic memory consolidation within the context of reinforcement learning [22], [23].

A similar approach, relying on continuous episodic memory, was proposed and tested across a range of OpenAI Gym environments [24] in the context of tasks involving sparse and delayed rewards. From the perspective of the decision-making process, this approach resonates with the works cited in [19], [20], as it entails utilizing the current context to retrieve the most relevant context from episodic memory for the purpose of selecting subsequent actions. Nevertheless, the use of discounted rewards in the aforementioned work still appears counterintuitive from a practical standpoint: actions performed during early stages—which eventually led to the receipt of the final reward—are subjected to heavier discounting than actions executed later, which merely episodically precede the moment of reinforcement. This contradicts the concept of "global feedback" [19], [20] according to which all states—including actions performed within a historical episode—are associated equally with the concluding positive or negative reinforcement.

The study [25] also approached the reinforcement learning problem from a neuro-physiological plausibility perspective by constructing a mathematical model simulating cortical columns for learning symbol-like representations that support interpretability and computational efficiency. The model was tested in the OpenAI Gym Atari "Pong" (a ping-pong game with an artificial opponent) and "Breakout" environments. In the latter environment, the maximum score was 420, and the average was 50 after 50,000 completed games (approximately 170 million steps).

## III. Computational Model and Architecture

The computational model used in our solution was based on the concept of a "vector model of psyche," based on earlier works by [20], [33], [34]. According to our interpretation of this concept, an agent perceives vectors $s_t$ of observations from the external environment and internal processes at each moment in time $t$, where $s_t$ consists of sensor observations $f_t$, internal motivations $y_t$, and awareness of completed actions $a_t$.

The agent maintains a historical memory of such state vectors as a sequence of $T$ states and can construct a model of the world's interactions as a transition graph between such sequences $\{s_\tau\}_{\tau\in\{-T,-1\}} => s_{\tau=0}$, where $\tau$ is the relative time for a given transition subgraph. Each transition in the model can be assigned its utility $U$ for satisfying the agent's motivations or needs, as well as a transition probability $P$ or a raw evidence count $C$ that determines this probability.

The learning function $L$ can compute the incremental update of utility $U$ when transitioning from the previous state at time $t-1$ to the new state at time $t$ as follows, where $x$ is the vector of importance of current motivations or needs, $y_t - y_{t-1}$ is the satisfaction or dissatisfaction assessment, interpreted as a "reward" in the context of reinforcement learning, corresponding to positive or negative "feedback" in our work. $E(a_{t-1})$ can be estimated as the amount of energy lost in case the agent seeks to minimize resource losses, but this was not considered in our work.

$$dU(\{s_\tau\}_{\tau\in\{-T,-1\}}, s_{\tau=0}) = L(x\cdot(y_t - y_{t-1}), E(a_{t-1})) \quad (1)$$

In this study, we considered $y$ as the desire to obtain scores satisfying the new metrics and the avoidance of "loss of life" with equal weights in $x$, so we used the simplified form.

$$L(y_t - y_{t-1}) = y_t - y_{t-1} \quad (2)$$

According to the "global feedback" principle proposed and evaluated in [19], the utility update applies to all transitions during the observation period that correspond to a significant historical episode or sequence of states between the moments of receiving positive (a brick is destroyed on the opposite side) or negative (losing a life during the game) feedback.

The decision making function can be based on maximizing the utility $U$ of transitioning from a given state sequence $\{s_\tau\}_{\tau\in\{-T,t-1\}}$ at time $t-1$ to a new state $s_t$ at time $t$, or on maximizing some function, in our study, the multiplication, combining the utility $U$ and the probability $P$ or the evidence count $C$. The choice was based on the "counted utility" (CU) hyper-parameter. That is, to select the next action, the most recent state sequence was searched in the model to determine the hypothetical transition point. The number of states to index during training and search was determined by the context size (CS) hyperparameter. If no state sequence identical to the current situation was found, the most similar state sequence could be found based on the cosine similarity between the actual and hypothetical state sequences exceeding the state similarity (SS) threshold hyperparameter. Then the expected next state $s'_t$ can be determined as follows, depending on PU policy. Once the expected state $s'_t$ is determined, the selected action vector $a'_t$ is extracted from it for execution.

$$\begin{aligned} &PU = False: \\ &s'_t = \arg\max_s U(\{s_\tau\}, s_{\tau=0}) \\ &\tau \in \{-T, -1\} \end{aligned} \quad (3)$$

$$\begin{aligned} &PU = True: \\ &s'_t = \arg\max_s U(\{s_\tau\}, s_{\tau=0}) \cdot C(\{s_\tau\}, s_{\tau=0}) \\ &\tau \in \{-T, -1\} \end{aligned} \quad (4)$$

In our study, the computational architecture included three layers: a state transformer, a state learning layer, and a decision making layer. The reference implementation, experimental setup, and presented results are available at *https://anonymous.4open.science/r/ICDM476D*.

### A. State Transformer

The state transformer was developed to process input data—specifically, a multidimensional grayscale pixel map image—and transform it into an interpretable representation

of objects and events specific to the given environment. Since the objective of this study was to verify the applicability of the "global feedback" principle—in the context of delayed and sparse feedback—within the OpenAI Gym Atari environment itself, the state transformer was designed as a separate, predefined component.

First, we computed the averaged grayscale background of an $N \times M$ pixel map based on a sliding window of the last $Z$ frames; then, for each frame, we subtracted this background from the current pixel map to isolate salient pixels. Next, we employed a two-projection Radon transform [35] to reduce the dimensionality of the $N \times M$ pixel matrix to $N+M$ sums, obtained through row-wise and column-wise summation. Based on this, the indices of the $N$ rows corresponding to maximum stacked values were interpreted as the vertical coordinates of respective objects, while the indices of the $M$ columns within those rows were treated as the horizontal coordinates of these objects.

In the case of the game Atari Breakout, these objects turned out to be the ball and the paddle. Moreover, our preliminary experiments demonstrated that knowledge of vertical coordinates does not contribute to mastering a winning strategy; therefore, it was excluded from consideration. That is, we utilized only the horizontal coordinates of individual pixel clusters as independent state variables, using them as elements of the vector $f_t$ to construct the state vector $s_t$. Other state elements included the action vector $a_t$ and the feedback vector $y_t$ (comprising positive and negative feedback variables)—provided that the "State reward" (SR) hyperparameter was specified, as described further.

### B. State Learner

The state learning component was based on an in-memory graph database capable of indexing transitions from some sequences of states to another states, where each state, sequence of states, and transition can be attributed with a learned utility and an evidence count. During interaction with the environment, the entire history of states $s_t$ was accumulated. When positive or negative feedback was detected, the current history of states at the time of receiving the feedback was remembered in the graph, such that each transition from $\{s_\tau\}_{\tau \in \{-T,-1\}}$ to $s_{\tau=0}$, where $T$ corresponds to the context size (CS, described further) hyperparameter, was memorized with a corresponding increase in the evidence count $C$ and an increase or decrease $dU$ in the utility $U$ in the case of positive or negative feedback, respectively, in accordance with the principle of "global feedback".

The implementation of the latter principle entailed that any experience of feedback—whether positive or negative—was utilized to form a sequential memory episode preceding that feedback, emotionally attributed based on either positive or negative value of the feedback, accordingly to [36]. In other words, while all transitions from a preceding series of $T$ states to a subsequent state were memorized through the accumulation of evidence, a complete log of all transitions between emotionally significant events oof feedback was preserved in a sequential episodic memory, and their utility was updated at the end of the episode based on the learning mode (LM), which determined precisely which type of feedback was taken into account.

### C. Decision Maker

The decision making component received the transformed state $s_t$ and combined it with previous states to construct a search key of a specified context size (CS=$T$); this enabled the retrieval of a corresponding entry from the transition graph—either via an exact match or based on the criterion of maximum similarity, as implemented in [24], provided that the similarity between the observed sequence of $T$ states and the closest matching state sequence in episodic memory exceeded a predefined state similarity threshold (SS). If none of the stored sequences of $T$ states matched, an attempt was made to match a sequence of $T-1$ states—and so on, until at least one state ($T = 1$) was matched with the context being experienced.

As soon as a suitable sequence of states—or a single state similar to one being experienced—was identified, a decision was made to transition into a new state with a corresponding action from among the available transition options based on either maximum utility $U$ alone or product $U \cdot C$ of utility and probability of the possible transitions, based on the "counted utility" (CU) hyperparameter. If no suitable sequence of states or transitions to a new state was found to determine the action, a random action was considered.

### D. Hyper-parameters

In accordance with the computational model described above, the following hyper-parameters were examined and evaluated in this study.

- Context size (CS): the number of subsequent states $T$ representing the current situation and indexed in the model in a state transition graph, where the sequence of $T$ (CS) states is considered as a key, and the next state is considered as a value to which the utility of the transition and its evidence count in the training history are assigned (default is CS=2)
- Learning mode (LM): 0 - no learning, 1 - learning on positive feedback only, 2 - learning on positive and negative feedback both (default is LM=2)
- State reward (SR): specifies if positive and negative rewards are included in the state ($True$, default) or not ($False$)
- Probable utility (PU): specifies the utility of the state transition is weighted by its evidence count value learned in the model ($True$) or not ($False$, default)
- Encode action (EA): specifies if the action is represented in the state as a one-hot encoded vector ($True$) or a scalar value ($False$, default)
- State count threshold (SC): specifies the minimum count of experiences of series of states (of size SC) to be considered as a reference for searching for possible transitions (default is SC=2)

- Similarity method (SM): specifies how to measure similarity between the states, such as either "cos" (cosine similarity), or "exp(-d)" ($exp(d)$), or "1/(1+d)" ($1/(1+d)$), or "1-d/max" ($1 - d/d_{max}$), where $d$ is the Euclidean distance between the states while $d_{max}$ is the "maximum corner distance" within the possible space of states
- State similarity threshold (SS): the minimum similarity of a series of states (of CS size) that can be considered applicable to use as the most similar state when searching for a possible transition if no ideal candidate for the current situation (of CS size) is found (default is SS=0.9, SS=1.0 means that only exact matching of states is considered)
- Transition utility threshold (TU): indicates the minimum utility of a transition in learning history for it to be considered as a candidate (default is TU=0)
- Transition count threshold (TC): indicates the minimum evidence count of a transition in learning history for it to be considered as a candidate (default is TC=1)
- Constant curiosity (CC): specifies the probability with which a random action is taken instead of a transition selected based on current state and transition utility (default is 0.0), which is the opposite to a "greediness" $\epsilon = 1 - CC$, so that CC=0 corresponds to the "greedy" strategy

## IV. Experiments

The experimental work in this study was carried out in three stages. In the first phase, we implemented a framework capable of "imitation learning" and assessed the ability of the framework to remember a history of states associated with positive or negative feedback, and then use these memories to act as if they had been learned through experience.

In the second phase, we conducted a cursory study of our framework's ability to learn from scratch without limit on the number of steps per game.

In the third phase, we performed a systematic study of the learnability of our framework for different values of the random seed and evaluated different combinations of hyper-parameters.

The OpenAI Gym environment for Breakout was initialized using the setup *gym.make('BreakoutNoFrameskip-v4', obs_type="grayscale")*, so that the frame skipping option was not set, and each frame was evaluated as a separate step, with a grayscale pixel array used for observations.

All experiments were conducted on low-cost computing equipment consisting of two laptops: 1) MSI Raider GE77HX 12UGS notebook with 12th Gen Intel(R) Core(TM) i7-12800HX 2.00 GHz, 32.0 GB RAM Laptop GPU running Windows; 2) MacBook Pro with 2.9 GHz 6-Core Intel Core i9, Radeon Pro 560X 4GB Intel UHD Graphics 630 1536 MB, 32 GB 2400 MHz DDR4 running MacOS. Python software version 3.11.13 was used.

The average performance during runtime evaluation was 100-200 steps or frames per second. Given 18,000 frames, corresponding to 5 minutes of game-play, and an average frame rate of 60 frames per second, real-time training during game-play could have been achieved. However, all evaluations were conducted with rendering disabled and no display output, so the speed of the experiments was 2-3 times higher than real-time. In reality, when running experiments in batches of 1,000 games, each batch took between 4 and 18 hours, depending on the number of actual steps in the game, with up to 7 instances of the evaluation environment running on a single laptop. The total time spent on the study, including computing resources, was approximately 6 weeks on both laptops.

### A. Phase 1: Exploring Decision Making Ability

First and foremost, we established an upper baseline—or "highest watermark" for our further experiments, based on the OpenAI Gym Atari Breakout environment. We implemented an "Automated" agent that knew the rules of the Breakout game. The agent received the ball and paddle coordinates from the pre-processing layer and executed actions to hit the ball based on its location. The resulting states, including the previous action that could influence the transition to that state, were stored in a state transition memory, and the every transition utility values were updated based on the received feedback. The "Automated" agent was tested in 100 games, which were played with an average score of 553 and a maximum score of 856, as shown in Fig. 1 (the "Automated playing" is shown in blue). The imperfect results (below 864) can be explained by the known "right wall" issue in the OpenAI Gym Breakout game, where the actual behavior of the paddle during play violates the assumed physical laws, breaking the wall when moving to the right.

Building upon the training phase described above—implemented using the "learning by imitation" method, wherein all actions of the "teacher" were captured within a graph-based decision-making model—we conducted an ablation study to confirm that our architecture enables the generation of accurate decisions based on an existing pre-trained model. A new "model-based" agent was exposed to play another 100 games, using the model learned by the previous agent, but without the opportunity to improve it during the game (LM=0), so the collected feedback was not used to update its memory. In this case, using the pre-learned model improved the maximum score to 860 (possibly due to randomness) with a slight decrease in the average score to 544, as shown in Fig. 1 ("Model-based playing, Learn=None").

The same agent using the same model was then run through 100 more games twice with different learning settings, as shown in Fig. 1: "Model-based playing, Learn=Pos" to account for only positive feedback (LM=1); "Model-based playing, Learn=Pos+Neg" to account for both positive and negative feedback (LM=2). In these runs, the agent was able to learn as it played, gradually improving the model as it played. With only positive feedback, the average score was 839, and the maximum score was 839. With both positive and negative feedback, the average score was 787, and the maximum score was the maximum possible score of 864.

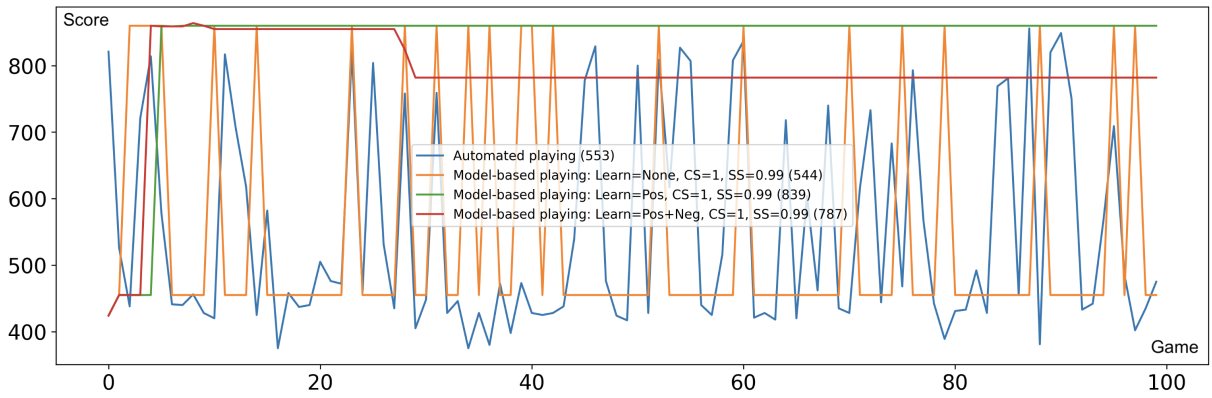


Fig. 1. Scores earned in four different runs playing 100 games. Horizontal axis - games from 1 to 100. Vertical axis - scores per game. Blue - "Automated" agent following the game rules based on pre-processed input providing tentative horizontal coordinates of the ball and the paddle. Orange - "Model-based" playing using the model pre-trained by "Automated" agent without the ability to learn. Green - "Model-based" playing using the same pre-trained model with learning based only on positive feedback. Red - "Model-based" playing using the same pre-trained model with learning based on both positive and negative feedback. Context size CS=1, state similarity threshold SS=0.99. Numbers in parentheses in the legend indicate average scores.

During the experiments described above, the context size was set to CS=1, so that only one state from history was used to search for a suitable situation and consider alternative transitions to new states. The state similarity threshold for searching for approximately suitable situations was set to SS=0.99. The initial value of the random number generator was not controlled, so completely random values based on the system time were used in all runs.

This ablation experiment confirmed the framework's ability to use a pre-learned (pre-trained) model to effectively perform actions (inference) with the ability to quickly improve it as it runs. Rapid performance improvements were observed in both training cases (with and without negative feedback) within the first ten games.

### *B. Phase 2: Exploring Learning Ability - Cursory*

The same framework was further cursory to briefly test the ability to learn from scratch. Testing was conducted using a "Model-based" agent across several different runs with different context size settings (CS=1, CS=2) and state similarity thresholds (SS=0.8, SS=0.9, SS=0.99, SS=0.999), as well as different learning modes (LM=1, LM=2). The initial value of the random number generator was not controlled, as in the previous phase. The evaluation was conducted in batches of 1000 games, with no limit on the number of steps per game.

The evaluation showed that achieving high results when training from scratch with CS=1 is impossible, although we observed improved performance with C=1 in the case of the pre-trained model presented in the subsection IV-A. This is clearly explained by the need to consider the direction of the ball's movement based on at least two consecutive states.

With a context size of two consecutive states (CS=2), the results obtained from the first 1,000 games were inconsistent. Most runs resulted in average scores ranging from 15 to 60 and maximum scores of up to 100, which still exceeded the human average score of 31. However, in some runs, we observed the ability to quickly master gaming skills, so that after several hundred games, the "Model-based" agent reached a score above 400, maintaining it consistently high with rare occasional dips. Moreover, for such high-performing ("quick learner") runs, where high scores were achieved within the first 1,000 games, up to 19 additional runs of 1,000 games were run re-using the same model, run by run. In this case, we observed stable performance at an average score above 400, gradually and slightly improving with occasional observation of top 864 scores.

Experiments conducted at this phase with different hyper-parameters confirmed the potential ability of the proposed algorithmic framework to learn from scratch, achieving a stable level of average performance outperforming (under certain random seeds) many known baseline models such as human (31), DQN (168) according to [5], Rainbow-IQN (176) according to [7], MARTI-5 (50) according to [25], and sometimes reaching the highest score of 864 achieved by MuZero [6].

The experiments confirmed the inappropriateness of using the context size CS=1 and proposed a context similarity (SS) threshold in the range of 0.9–0.99 with a transition utility threshold TU=0, which was used as the initial hyper-parameter setting in the next stage of experiments.

### *C. Phase 3: Exploring Learning Ability - Systematic*

Based on the cursory exploration using uncontrolled random seeds described above, a final phase was to conduct a systematic reproducible study using different fixed random seeds, the results of which are presented in Fig. 2, Fig. 3, and Fig. 4.

First, 30 different random seed values were evaluated using the default hyper-parameters (LM=2, TU=0, CS=2, TC=1, SC=2, SR=True, SS=0.9, CU=$False$, EA=$False$) according to the results of the previous study. The evaluation was

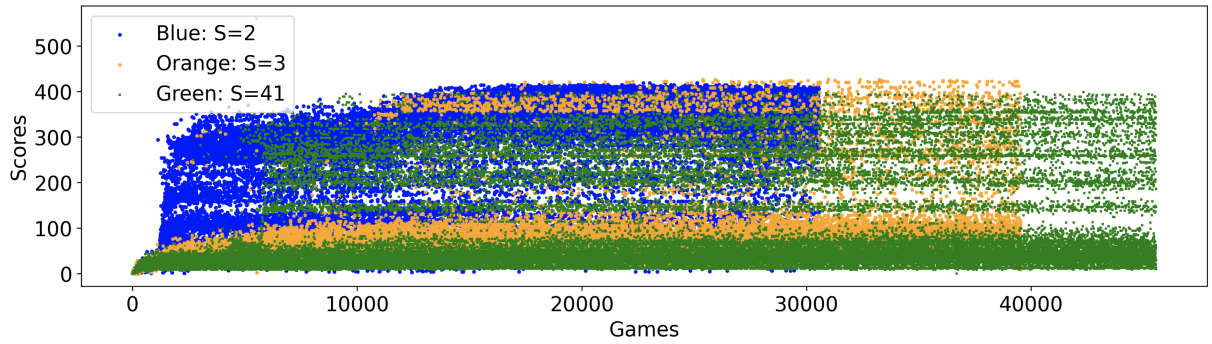


Fig. 2. Scores obtained in three different runs with different random seeds, for 200 million frames (steps) with three different fixed random seeds for the state similarity threshold SS=1.0, context size CS=2. Horizontal axis - numbers of games completed during the each of the runs. Vertical axis - scores per game. Scatter points of different colors correspond to different random seeds S (blue – S=2, orange – S=3, green – S=41).

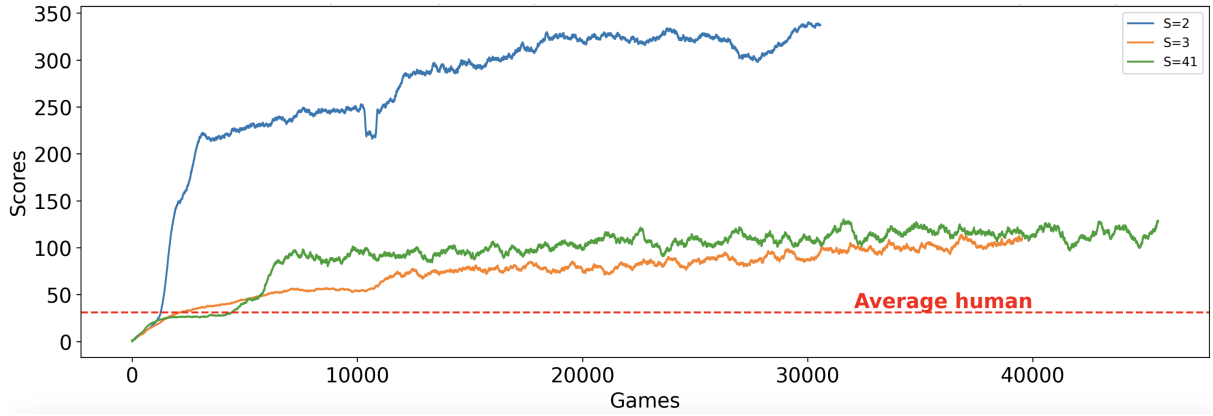


Fig. 3. Average scores in a sliding window of last 500 games across three different runs for 200 million frames (steps) with three different fixed random seeds for the state similarity threshold SS=1.0, context size CS=2. Horizontal axis - numbers of games completed during the each of the runs. Vertical axis - mean scores per game. Plots in different colors correspond to different random seeds S (blue – S=2, orange – S=3, green – S=41).

conducted based on 1000 games, and the average scores were collected, with the resulting average score values ranging from 3 to 30.

Based on this, the best performing random seed value of 41 was chosen to search for the optimal values of the state similarity threshold hyper-parameter. Runs were run with all other hyper-parameters set to their default values and SS values of 0.75, 0.8, 0.85, 0.9, 0.95, 0.99, and 0.999, using SM="cos". Based on that, SS=0.9 and SS=0.95 were found to be the optimal values based on the average values across all 1000 games (25.8 and 31, respectively). Based on SS=0.9 being the best value in both the previous and present phases of the study, a search was conducted for the other hyper-parameters discussed in the subsection III-D. The results confirmed the default hyper-parameter values determined in the previous cursory study, as stated further along with all explored options.

Based on the distribution of scores for the 30 seeds, 7 representative seeds were selected for the validation experiment, and another evaluation was performed on these 7 seeds using 1000 games with state similarity thresholds of SS=0.9 and SS=0.95 (SM="cos"), using the default hyper-parameters. Considering the recent results for these 7 seeds and 2 similarity thresholds, we selected three seeds (S) that yielded the highest mean score (S=41), the lowest mean score (S=2), and the score between the highest and the lowest (S=3) for further investigation.

Using the three selected fixed random seeds (2, 3, 41), we conducted a final re-evaluation of all hyper-parameters. Since the earlier baseline studies [5] and [7] based their results on frame count rather than the number of games, as we reported above, further exploration was based on frame count. The hyper-parameter search in this round was conducted for all three random seeds with the number of frames set to 1,080,000 (corresponding to 5 hours of gameplay). The best hyper-parameters were determined based on the average game score obtained over the entire gameplay.

During the latter re-evaluation, we first found that

SM="cos" was not optimal, so the best results, independent of the SS setting, were obtained using SM="exp(-d)". Furthermore, we found that the best and most stable results for all three random seeds were obtained with SS=1.0, so the similarity method was no longer relevant. This also changed the choice of the most effective random seeds, so that S=2, instead of s=41, yielded the highest average scores for the final set of hyper-parameters specified below. Additionally, processing time-efficiency with SS=1.0 was greatly improved because similarity calculations were not involved, so that each run on 1,080,000 frames with SS=1.0 took 5 minutes instead of several hours (1.5 to 9.5 hours) for other state similarity thresholds.

The final set of optimal hyper-parameters is provided below.

- CS=2 (context size, values: 1, 2, and 3)
- LM=2 (learning mode: 0 - no learning, 1 - only from positive feedback, 2 - from both positive and negative feedback)
- SR=True (state reward, values: $True, False$)
- PU=False (counted utility, values: $True, False$)
- EA=False (encode action, values: $True, False$)
- SC=2 (state count threshold, values: 1, 2, 3)
- SM="exp(-d)" (similarity method, values: "cos", "exp(-d)", "1/(1+d)", "1-d/max") - did not affect the final results, as SS=1.0 was chosen so that only exact matches were counted.
- SS=1.0 (state similarity threshold, values: 0.75, 0.8, 0.85, 0.9, 0.95, 0.99, 0.999, 1.0)
- TU=0 (transition utility threshold, values: $None$, 0, 1, 2)
- TC=1 (transition count threshold, values: 1, 2, 3)
- CC=0.0 (constant curiosity, values: 0.001, 0.01, 0.1, 0.2, 0.8, 0.9, 0.99)

To evaluate the learning ability of our system by comparing its performance with earlier results presented on the basis of game steps or frames according to [5] and [7], we further tested our solution using a specific hyperparameter configuration with the same number of frames as reported in [7] (10, 50, 100, and 200 million).

For each umber of frames, we ran a learning experiment with the same random seeds as shown in Fig. 2 and calculated the average ratings for the last 500 games from each run according to [25], as shown in Fig. 3. The results of all experiments are presented in Fig. 4.

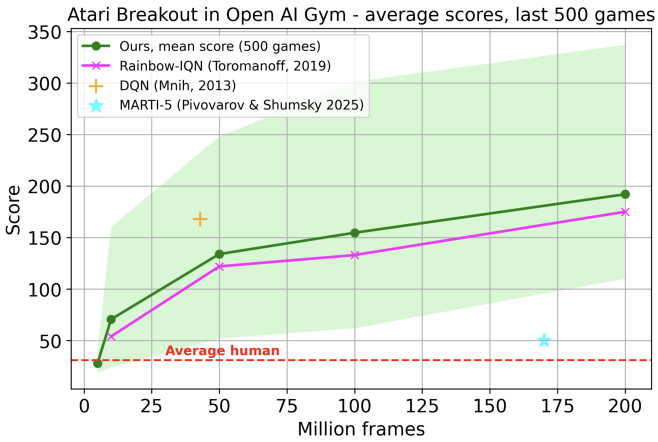


Fig. 4. Comparison of the results obtained in different runs of our system with different random seeds (2, 3, 41) with the results obtained in [5] and [7], depending on the number of frames spent on training. The dark green plot corresponds to the average of the average values for different random seeds over the last 500 games per each run; the light green area corresponds to the minimum and maximum for three runs with the corresponding seeds.

## V. Discussion

### A. Interpretation and Comparison with Prior Art

A key observation is that, for certain random seeds, we can achieve the same or better learning performance for the Atari Breakout game in OpenAI Gym based on the number of training frames compared to known baseline approaches based on DQN [5] and Rainbow-IQN [7] or the neuro-physiologically plausible design in MARTI-5 [25]. Our average performance across all three seeds is consistently higher, for any number of training frames, than reported in [7] and [25].

As reported in [5], the DQN experiment achieved an average score of 168 at approximately 5,400,000 frames, however the experiment involved randomly sampling 32 experiences from the replay buffer at each step, with each step taken after 4 frames. That is, we adjust the number by assuming that each frame in the history was exposed to the system 32 times every fourth frame, so the effective number of training frames was approximately 43 million. This may also mean that the reported latter performance may be higher than our average for a similar number of training frames (50 million) due to overfitting because of the implicit re-use of data samples.

The only interpretable alternative, MARTI-5 [25], showed an average score of 50 after training on 170 million training frames below our lowest scores for the same number of frames.

We observe that our system can surely surpass human performance (a score of 31) after of 25 million frames and further compete with alternative approaches based on DQN and IQN steadily with different volumes of training data, at a speed of 3,600 frames per second. We attribute the effectiveness of our solution to the "global feedback" principle [19] and computational efficiency direct matching of the states and sequences of states of used in our system given the latest "winning" set of hyper-parameters with SS=1.0.

We note that our system can clearly outperform human performance (a score of 31) after processing 25 million frames and further consistently compete with alternative DQN-based and IQN-based approaches across various training data sizes, at 3600 frames per second. We attribute the performance of our solution to the "global feedback" principle [19] and the computational efficiency of the direct matching between states and state sequences used in our system, given the final "winning" hyperparameter set with SS=1.0.

This performance was achieved using a model consisting of a transparent state transition graph based on a history of states that does not include hidden or latent variables or states,

which is fully interpretable and satisfies both local and global interpretability conditions [37]. Local interpretability means that any action recommended by the framework in a given situation can be traced back to a prior state transition, as well as to the known values of these states and transitions and alternative options in a given situation. All of this can be found in the model, which represents historical transition graphs at a given context depth. Global interpretability implies that transition graphs can be used for process mining [10]–[12] and causal analysis [14] to identify meaningful patterns and understand the model's behavior.

### B. Space and Time Analysis

In the case of the Atari Breakout, the dimensionality of the initial state—represented by a 160-row by 210-column RGB pixel map—was initially reduced by converting the image to grayscale, subsequently by isolating a 144-row by 178-column screen region relevant to the game's dynamics. The further dimensionality reduction to the coordinates of moving pixel objects condensed the state representation to two coordinates for the paddle and one coordinate for the ball, supplemented by three variables encoding feedback and the executed action.

If one multiplies the number of possible positions of the ball and the racket, the number of distinct actions, and the possible combinations of feedback variable values, the resulting potential state space amounts to about 55 million possible states.

It was found that across all experimental runs, the number of known states grew rapidly, reaching approximately 20,000 states within the first 500 games. Subsequently, this growth slowed significantly, and the number approached 35,000 states (at 200 million frames, corresponding to 30,000-45,000 games) semi-asymptotically. Thus, the state space compression ratio for a well-trained agent in our experiments was approximately 1500.

Under the described conditions, memory consumption did not exceed 300 MB (11 KB per state), and the game speed in "human" rendering mode corresponded to real-time, while the experiment in "headless" mode ran significantly faster than real-time (3600 frames per second in case of direct matching only with SS=1.0).

### C. Limitations of the approach

Our solution currently lacks the implementation of multi-agency, motivation for exploratory activity and planning features present in [4], [6], [26], [28], which may explain why our results are below the baseline presented in these studies.

The pre-processing required by our current implementation to reduce the state dimensionality and make it game-specific may be not considered universal and may provide an unfair competitive advantage when compared to other baselines based on raw pixels without manual feature engineering.

Another interesting observation and potential for future study is that the "greedy strategy" (CC=0.0 or $\epsilon$=1.0) turned out to be the best in our experiments, so in the future we may want to implement a "conditional curiosity" ("conditional greediness") based on the discrepancy between the agent's expectations about the expected state $s'_t$ and the actual experience in this state after the next step.

## VI. Conclusion

We proposed, implemented, and evaluated an interpretable architecture for experiential and reinforcement learning, based on a model consisting of weighted transition graphs between a series of successor states, with complex weights corresponding to the utility and evidence count of these transitions, which enables efficient process mining applications and causal analytics. The architecture was successfully tested using the OpenAI Gym Atari Breakout environment, showing competitive results against several baselines such as DQN and IQN-based models, and outperforming them in certain conditions when using low-end computing environments.

### A. Impact Statement

The proposed approach has the potential to enable the development of interpretable solutions using reinforcement learning and experiential learning principles for robust applications in mission-critical and resource-constrained computing environments such as industrial automation, energy, home automation, embedded systems, and edge computing.

Representing a model as a weighted transition graph between interpretable state sequences with state variables corresponding to real-world objects and events allows for its verification, validation, and adjustment. Graph transition weights, assigned based on the utility and evidence counts, can make model analysis more practical and informative. Linking state transition evidence count data to system logs can also make the model fully auditable.

Experimental evaluation of the model on the OpenAI Gym Atari Breakout benchmark confirmed its ability to run in real time on low-end computing infrastructure with competitive performance. Preservation of the evidence counts in the model allows for model compression based on the evidence count thresholds, making the model even more efficient in terms of execution speed and response time.

### B. Future Work

In future work, we plan to improve the stability of our architecture's learning performance, expand its applicability to various environments, and explore the possibility of redesigning the pre-processing layer for learning the generalized interpretable representation to improve scalability and generalizability. We will also consider extending our architecture to support multi-agency, exploratory activity and planning capacity to achieve higher scores, while continuing to build on our interpretable and resource-efficient approach.

## Acknowledgment

This work was supported by a grant for Anonymized, provided by the Anonymized in accordance with the subsidy agreement with the Anonymized.

## REFERENCES


[1] Alginahi, Y. M., Sabri, O., and Said, W. "Reinforcement learning for industrial automation: A comprehensive review of adaptive control and decision-making in smart factories", *Machines*, 13(12), 2025. ISSN 2075-1702. https://www.mdpi.com/2075-1702/13/12/1140.

[2] Oh, J., Farquhar, G., Kemaev, I., Calian, D. A., Hessel, M., Zintgraf, L., Singh, S., van Hasselt, H., and Silver, D. "Discovering state-of-the-art reinforcement learning algorithms". *Nature*, 648(8093):312–319, 2025. https://doi.org/10.1038/s41586-025-09761-x.

[3] Yang, G., Chen, Y., Yen, T., and Namkoong, H. "Benchmarking in-context experiential learning through repeated product recommendations", 2025. arXiv:2511.22130 [cs.LG]. https://arxiv.org/abs/2511.22130.

[4] Badia, A. P., Piot, B., Kapturowski, S., Sprechmann, P., Vitvitskyi, A., Guo, D., and Blundell, C. "Agent57: Outperforming the atari human benchmark", 2020. arXiv:2003.13350 [cs.LG]. https://arxiv.org/abs/2003.13350.

[5] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. "Playing atari with deep reinforcement learning", 2013. arXiv:1312.5602 [cs.LG]. https://arxiv.org/abs/1312.5602.

[6] Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., and Silver, D. "Mastering atari, go, chess and shogi by planning with a learned model", *Nature*, 588(7839):604–609, December 2020. ISSN 1476-4687. http://dx.doi.org/10.1038/s41586-020-03051-4.

[7] Toromanoff, M., Wirbel, E., and Moutarde, F. "Is deep reinforcement learning really superhuman on atari Leveling the playing field", 2019. arXiv:1908.04683 [cs.AI]. https://arxiv.org/abs/1908.04683.

[8] Vouros, G. A. "Explainable deep reinforcement learning: State of the art and challenges", *ACM Comput. Surv.*, 55(5), December 2022. ISSN 0360-0300. https://doi.org/10.1145/3527448.

[9] Xin, Q., Wu, G., Fang, W., Cao, J., and Ping, Y. "Opportunities for reinforcement learning in industrial automation", In *2021 7th International Conference on Big Data and Information Analytics (BigDIA)*, pp. 496–504, 2021. DOI 10.1109/BigDIA53151.2021.9619637.

[10] Dogan O, Areta Hiziroglu O. "Empowering Manufacturing Environments with Process Mining-Based Statistical Process Control", Machines. 2024; 12(6):411. https://doi.org/10.3390/machines12060411

[11] Koschmider, A., Aleknonytė-Resch, M., Fonger, F., Imenkamp, C., Lepsien, A., Apaydin, K., Janssen, D., Langhammer, D., Ziolkowski, T., and Zisgen, Y. "Process mining for unstructured data: Challenges and research directions", In *Modellierung 2024*, pp. 119–136. Gesellschaft für Informatik e.V., Bonn, 2024. ISBN 978-3-88579-742-5. DOI 10.18420/modellierung2024_012.

[12] Khan, A., Ghose, A., Dam, H., and Syed, A. "Advances in process optimization: A comprehensive survey of process mining, predictive process monitoring, and process-aware recommender systems", 2025. arXiv:2301.10398 [cs.OH]. https://arxiv.org/abs/2301.10398.

[13] Kolonin, A. "Representing scenarios for process evolution management", 2018. arXiv:1807.02072 [cs.AI] https://doi.org/10.48550/arXiv.1807.02072

[14] Van Houdt, G., Depaire, B., Martin, N. "Root Cause Analysis in Process Mining with Probabilistic Temporal Logic", 2022. In: Munoz-Gama, J., Lu, X. (eds) Process Mining Workshops. ICPM 2021. Lecture Notes in Business Information Processing, vol 433. Springer, Cham. https://doi.org/10.1007/978-3-030-98581-3_6

[15] Christopoulos, M., Spantideas, S., Giannopoulos, A., and Trakadas, P. "Deep reinforcement learning for smart home temperature comfort in iot-edge computing systems", In *Proceedings of the 1st International Workshop on MetaOS for the Cloud-Edge-IoT Continuum*, MECC '24, pp. 1–7, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705434. https://doi.org/10.1145/3642975.3678961.

[16] Latoń, D., Grela, J., and Ożadowicz, A. "Applications of deep reinforcement learning for home energy management systems: A review", *Energies*, 17(24), 2024. ISSN 1996-1073. https://www.mdpi.com/1996-1073/17/24/6420.

[17] Sen, A. P., Goyal, M. K., and Shalini. "Deploying reinforcement learning approaches for smart home automation", In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–5, 2024. DOI 10.1109/ICCCNT61001.2024.10724603.

[18] Farooq, A. and Iqbal, K. "A survey of reinforcement learning for optimization in automation", In *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, pp. 2487–2494. IEEE, August 2024. URL: http://dx.doi.org/10.1109/CASE59546.2024.10711718.

[19] Kolonin, A. "Neuro-symbolic architecture for experiential learning in discrete and functional environments", In Goertzel, B., Iklé, M., and Potapov, A. (eds.), *Artificial General Intelligence*, pp. 106–115, Cham, 2022. Springer International Publishing. ISBN 978-3-030-93758-4. DOI 10.1007/978-3-030-93758-4_12.

[20] Kolonin, A. and Kryukov, V. "Computational Concept of the Psyche", 2025. arXiv:2603.15586 [cs.AI] https://arxiv.org/abs/2603.15586.

[21] Bengio, Y., Courville, A., and Vincent, P. "Representation learning: A review and new perspectives", *IEEE Trans. Pattern Anal. Mach. Intell.*, 35(8):1798–1828, August 2013. ISSN 0162-8828. DOI 10.1109/TPAMI.2013.50. https://doi.org/10.1109/TPAMI.2013.50.

[22] Lee J. W., Jung M.W. "Memory consolidation from a reinforcement learning perspective", Front Comput Neurosci. 2025 Jan 8;18:1538741. DOI 10.3389/fncom.2024.1538741. PMID 39845091; PMCID PMC11751224.

[23] Gershman S.J., Daw N.D. "Reinforcement Learning and Episodic Memory in Humans and Animals: An Integrative Framework", Annu Rev Psychol. 2017 Jan 3;68:101-128. DOI 10.1146/annurev-psych-122414-033625. Epub 2016 Sep 2. PMID 27618944; PMCID PMC5953519.

[24] Yang Z., Moerland T. M., Preuss M., Plaat A. "Continuous Episodic Control", 2022. arXiv:2211.15183 [cs.LG] https://arxiv.org/abs/2211.15183.

[25] Pivovarov, I. and Shumsky, S. "Marti-5: A mathematical model of self in the world as a first step toward self-awareness", 2025. arXiv:2512.10985 [q-bio.NC]. https://arxiv.org/abs/2512.10985.

[26] Kapturowski, S., Ostrovski, G., Quan, J., Munos, R., and Dabney, W. "Recurrent experience replay in distributed reinforcement learning", In *International Conference on Learning Representations (ICLR) 2019*, 2019. Poster session (ICLR). https://openreview.net/forum?id=r1lyTjAqYX.

[27] Anand, A., Racah, E., Ozair, S., Bengio, Y., Côté, M.-A., and Hjelm, R. D. "Unsupervised state representation learning in Atari", chapter 787, pp. 8769 – 8782. Curran Associates Inc., Red Hook, NY, USA, 2019.

[28] Badia, A. P., Sprechmann, P., Vitvitskyi, A., Guo, D., Piot, B., Kapturowski, S., Tieleman, O., Arjovsky, M., Pritzel, A., Bolt, A., and Blundell, C. "Never give up: Learning directed exploration strategies", 2020. arXiv:2002.06038 [cs.LG]. https://arxiv.org/abs/2002.06038.

[29] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. "The arcade learning environment: an evaluation platform for general agents", *J. Artif. Int. Res.*, 47(1):253–279, May 2013. ISSN 1076-9757.

[30] Glanois, C., Weng, P., Zimmer, M., Li, D., Yang, T., Hao, J., and Liu, W. "A survey on interpretable reinforcement learning", 2022. arXiv:2112.13112 [cs.LG]. https://arxiv.org/abs/2112.13112.

[31] Zhao, C., Deng, C., Liu, Z., Zhang, J., Wu, Y., Wang, Y., and Yi, X. "Interpretable reinforcement learning of behavior trees", In *Proceedings of the 2023 15th International Conference on Machine Learning and Computing*, ICMLC '23, pp. 492–499, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450398411. DOI: 10.1145/3587716.3587798. URL: https://doi.org/10.1145/3587716.3587798.

[32] Paleja, R., Chen, L., Niu, Y., Silva, A., Li, Z., Zhang, S., Ritchie, C., Choi, S., Chang, K. C., Tseng, H. E., Wang, Y., Nageshrao, S., and Gombolay, M. "Interpretable reinforcement learning for robotics and continuous control", 2023. URL: https://arxiv.org/abs/2311.10041.

[33] Kotliarov, I. "Mathematical theory of labour motivation", *Ekonomija Economics*, 13(2):393–408, None 2007. URL: https://ideas.repec.org/a/eff/ekoeco/v13y2007i2p393-408.html.

[34] Petrenko, V. F. and Suprun, A. P. "Goal-oriented systems, evolution, and the subjective aspect in systemology", volume 62, chapter 1. 2012. Institute of System Analysis, RAS, Moscow.

[35] Radon, J. "Über die Bestimmung von Funktionen durch ihre Integralwerte längs gewisser Mannigfaltigkeiten", 1917. Berichte über die Verhandlungen der Königlich Sächsischen Gesellschaft der Wissenschaften zu Leipzig, Mathematisch-Physische Klasse, 69, 262-277.

[36] Simonov, P. V. "The Emotional Brain: Physiology, Neuroanatomy, Psychology, and Emotion", 1986. Springer, ISBN 978-0-306-42363-5

[37] Bassan, S., Amir, G., and Katz, G. "Local vs. global interpretability: A computational complexity perspective", 2024. arXiv:2406.02981 [cs.LG]. https://arxiv.org/abs/2406.02981.